Resolución — Taller SQL sobre HR 1 — Filtros, texto y NULL

Bases de Datos I · resolución paso a paso

Antes de empezar

- Cada ejercicio muestra lo que se pide, la idea, los pasos, la consulta y un aviso de error común.
- Las consultas están escritas para Oracle Database 19c y solo leen el esquema HR.
- Hay más de una forma correcta de resolver casi todos los ejercicios; cuando conviene, se muestra una variante.

Ejercicio 1 — Empleados con comisión

Nivel: Básico · **Tablas:** `employees`

Se pide. Traer todos los empleados cuya comisión sea diferente de nulo.

Mostrar: nombre, apellido y porcentaje de comisión.

Idea. `NULL` significa «sin dato», no «cero». Por eso no se compara con = ni con <>: se pregunta con `IS NULL` o `IS NOT NULL`.

Pasos.

1. La comisión vive en la columna `commission_pct` de `hr.employees`; no hace falta ningún `JOIN`.
2. El filtro es `commission_pct IS NOT NULL`.
3. En el `SELECT` van solo las tres columnas pedidas.

```
SELECT e.first_name,
       e.last_name,
       e.commission_pct
FROM hr.employees e
WHERE e.commission_pct IS NOT NULL;
```

Ojo. `commission_pct <> NULL` y `commission_pct != NULL` no dan error, pero devuelven cero filas: cualquier comparación con `NULL` no es verdadera.

Ejercicio 2 — Salario dentro de un rango

Nivel: Básico · **Tablas:** `employees`

Se pide. Listar los empleados del departamento 50 cuyo salario esté entre 2500 y 3500, incluyendo ambos extremos, del salario más alto al más bajo.

Mostrar: nombre, apellido y salario.

Idea. `BETWEEN a AND b` es una forma corta de escribir `>= a AND <= b`.

Pasos.

1. Las dos condiciones salen de `hr.employees`: `department_id = 50` y `salary BETWEEN 2500 AND 3500`.
2. Se unen con `AND`, porque el empleado debe cumplir ambas.
3. `ORDER BY e.salary DESC` ordena del mayor al menor.

```
SELECT e.first_name,
       e.last_name,
       e.salary
FROM hr.employees e
WHERE e.department_id = 50
  AND e.salary BETWEEN 2500 AND 3500
ORDER BY e.salary DESC;
```

Ojo. El valor menor va primero: `BETWEEN 3500 AND 2500` no da error, pero tampoco devuelve ninguna fila.

Ejercicio 3 — Empieza por S y termina en n

Nivel: Básico · **Tablas:** `employees`

Se pide. Listar los empleados cuyo nombre empiece por «S» y cuyo apellido termine en «n».

Mostrar: nombre y apellido, ordenados por apellido.

Idea. En `LIKE`, el símbolo `%` reemplaza cualquier cantidad de caracteres (incluso ninguno) y `_` reemplaza exactamente uno.

Pasos.

1. Nombre que empieza por S: `first_name LIKE 'S%'`.
2. Apellido que termina en n: `last_name LIKE '%n'`.
3. Las dos condiciones se unen con `AND` y se ordena por `last_name`.

```
SELECT e.first_name,
       e.last_name
FROM hr.employees e
WHERE e.first_name LIKE 'S%'
  AND e.last_name LIKE '%n'
ORDER BY e.last_name;
```

Ojo. En Oracle `LIKE` distingue mayúsculas de minúsculas: `'S%'` no encuentra «sandra». Si no sabes cómo está escrito el dato, compara contra `UPPER(first_name) LIKE 'S%'`.

Ejercicio 4 — Apellidos de exactamente cinco letras

Nivel: Intermedio · **Tablas:** `employees`

Se pide. Listar los empleados cuyo apellido tenga exactamente cinco letras.

Mostrar: apellido y su longitud, ordenados alfabéticamente.

Idea. `LENGTH(texto)` cuenta los caracteres; compararlo con un número da el filtro.

Pasos.

1. La longitud se calcula con `LENGTH(e.last_name)`.
2. El filtro es `LENGTH(e.last_name) = 5`.
3. La longitud también se muestra como columna, con el alias `letras`.

```
SELECT e.last_name,
       LENGTH(e.last_name) AS letras
FROM hr.employees e
WHERE LENGTH(e.last_name) = 5
ORDER BY e.last_name;
```

Ojo. `LENGTH` cuenta espacios y guiones como caracteres: un apellido compuesto suma más de lo que parece.

Variante. El mismo filtro con comodines: `WHERE e.last_name LIKE '_____'` (cinco guiones bajos).

Ejercicio 5 — Correo corporativo

Nivel: Intermedio · **Tablas:** `employees`

Se pide. Generar el correo corporativo de los empleados del departamento 90 con el formato `usuario@empresa.com`, en minúsculas, donde `usuario` es la columna `email`.

Mostrar: nombre, apellido y correo corporativo.

Idea. `||` concatena texto en Oracle y `LOWER` pasa todo a minúsculas.

Pasos.

1. Los empleados salen de `hr.employees` filtrando `department_id = 90`.
2. El correo se arma con `LOWER(e.email) || '@empresa.com'`.
3. Se le pone el alias `correo_corporativo`.

```
SELECT e.first_name,
       e.last_name,
       LOWER(e.email) || '@empresa.com' AS correo_corporativo
FROM hr.employees e
WHERE e.department_id = 90;
```

Ojo. `+` no concatena texto en Oracle: `e.email + '@empresa.com'` falla con `ORA-01722` (número no válido).

Ejercicio 6 — Nombre en formato de lista

Nivel: Intermedio · **Tablas:** `employees`

Se pide. Mostrar los empleados del departamento 60 con el nombre escrito como «APELLIDO, Nombre»: apellido en mayúsculas y nombre con la primera letra en mayúscula. Ordenar alfabéticamente.

Mostrar: una sola columna llamada `empleado`.

Idea. `UPPER` pasa todo a mayúsculas e `INITCAP` pone en mayúscula la primera letra de cada palabra.

Pasos.

1. Se filtra `department_id = 60`.
2. Se arma el texto con `UPPER(e.last_name) || ', ' || INITCAP(e.first_name)`.
3. `ORDER BY empleado` ordena por el alias: en `ORDER BY` sí se puede usar el alias de columna.

```
SELECT UPPER(e.last_name) || ', ' || INITCAP(e.first_name) AS empleado
FROM hr.employees e
WHERE e.department_id = 60
ORDER BY empleado;
```

Ojo. El alias solo se puede usar en `ORDER BY`; en `WHERE` y `GROUP BY` hay que repetir la expresión.

Ejercicio 7 — Teléfonos con guiones

Nivel: Intermedio · **Tablas:** `employees`

Se pide. De los empleados cuyo teléfono empieza por «515», mostrar el teléfono reemplazando cada punto por un guion.

Mostrar: nombre, apellido y teléfono con guiones.

Idea. `REPLACE(texto, buscar, reemplazo)` cambia todas las apariciones del texto buscado.

Pasos.

1. El filtro es `phone_number LIKE '515%'`.

2. La transformación es `REPLACE(e.phone_number, '.', '-')`; en `REPLACE` el punto es un carácter normal, no un comodín.
3. Se le pone el alias `telefono`.

```
SELECT e.first_name,
       e.last_name,
       REPLACE(e.phone_number, '.', '-') AS telefono
FROM hr.employees e
WHERE e.phone_number LIKE '515%';
```

Ojo. En `LIKE` el punto tampoco es comodín: los únicos comodines son `%` y `_`.

Ejercicio 8 — Los cinco empleados más antiguos

Nivel: Intermedio · **Tablas:** `employees`

Se pide. Mostrar los cinco empleados con más antigüedad, es decir, los primeros contratados.

Mostrar: nombre, apellido y fecha de contratación con el formato DD/MM/AAAA.

Idea. Ordenar por fecha ascendente deja primero la más antigua, y `FETCH FIRST` corta el resultado.

Pasos.

1. Se ordena por la fecha real: `ORDER BY e.hire_date`.
2. La fecha se formatea solo en el `SELECT`, con `TO_CHAR(e.hire_date, 'DD/MM/YYYY')`.
3. `FETCH FIRST 5 ROWS ONLY` va al final de la consulta.

```
SELECT e.first_name,
       e.last_name,
       TO_CHAR(e.hire_date, 'DD/MM/YYYY') AS fecha_contratacion
FROM hr.employees e
ORDER BY e.hire_date
FETCH FIRST 5 ROWS ONLY;
```

Ojo. Si ordenaras por el alias `fecha_contratacion` (ya convertida a texto), el orden sería alfabético: «01/…» antes que «02/…» sin importar el año. Ordena siempre por la columna original.

Variante. `FETCH FIRST 5 ROWS WITH TIES` incluye también a quienes empatan con el quinto.

Ejercicio 9 — Contratados en junio, julio o agosto

Nivel: Intermedio · **Tablas:** `employees`

Se pide. Listar los empleados que fueron contratados en junio, julio o agosto de cualquier año.

Mostrar: nombre, apellido y fecha de contratación (DD/MM/AAAA), ordenados por fecha.

Idea. `EXTRACT(MONTH FROM fecha)` devuelve el número del mes, y `IN` compara contra una lista de valores.

Pasos.

1. El mes se extrae con `EXTRACT(MONTH FROM e.hire_date)`.
2. `IN (6, 7, 8)` equivale a tres comparaciones unidas por `OR`.
3. Se muestra la fecha formateada y se ordena por la fecha real.

```
SELECT e.first_name,
       e.last_name,
       TO_CHAR(e.hire_date, 'DD/MM/YYYY') AS fecha_contratacion
FROM hr.employees e
WHERE EXTRACT(MONTH FROM e.hire_date) IN (6, 7, 8)
ORDER BY e.hire_date;
```

Ojo. `EXTRACT(MONTH FROM ...)` devuelve un número (6), no el nombre del mes.

Variante. `TO_CHAR(e.hire_date, 'MM') IN ('06', '07', '08')` hace lo mismo comparando texto.

Ejercicio 10 — Sin departamento o sin jefe

Nivel: Intermedio · **Tablas:** `employees`

Se pide. Encontrar los empleados que no tienen departamento asignado o que no tienen jefe.

Mostrar: nombre, apellido, `department_id` y `manager_id`.

Idea. Dos condiciones con `IS NULL` unidas por `OR`: basta con que se cumpla una.

Pasos.

1. Sin departamento: `department_id IS NULL`.
2. Sin jefe: `manager_id IS NULL`.
3. Se unen con `OR`; con `AND` solo saldría quien carezca de las dos cosas a la vez.

```
SELECT e.first_name,
       e.last_name,
       e.department_id,
       e.manager_id
FROM hr.employees e
WHERE e.department_id IS NULL
   OR e.manager_id IS NULL;
```

Ojo. `department_id = NULL` nunca es verdadero: para preguntar por un `NULL` siempre se usa `IS NULL`.

Ejercicio 11 — Dos letras «a» en el nombre

Nivel: Avanzado · **Tablas:** `employees`

Se pide. Listar los empleados cuyo nombre tenga al menos dos letras «a», sean mayúsculas o minúsculas.

Mostrar: nombre, apellido y cuántas «a» tiene el nombre; primero quienes más tienen y, en empate, por nombre.

Idea. No existe una función que cuente letras, pero se puede medir cuánto se acorta el texto al borrar la letra: largo original menos largo sin las «a».

Pasos.

1. `LOWER(e.first_name)` unifica mayúsculas y minúsculas.
2. `REPLACE(..., 'a', '')` borra todas las «a».
3. La diferencia de longitudes es la cantidad de «a» que había.
4. Esa cuenta se usa en el `SELECT`, en el `WHERE` (`>= 2`) y en el `ORDER BY`.

```
SELECT e.first_name,
       e.last_name,
       LENGTH(e.first_name)
         - LENGTH(REPLACE(LOWER(e.first_name), 'a', '')) AS cantidad_a
FROM hr.employees e
WHERE LENGTH(e.first_name)
        - LENGTH(REPLACE(LOWER(e.first_name), 'a', '')) >= 2
ORDER BY cantidad_a DESC, e.first_name;
```

Ojo. El alias `cantidad_a` no se puede usar en el `WHERE`: se repite la expresión o se envuelve la consulta en una vista en línea.

Variante. `REGEXP_COUNT(e.first_name, 'a', 1, 'i')` cuenta las coincidencias sin importar mayúsculas, y se usa igual: `WHERE REGEXP_COUNT(e.first_name, 'a', 1, 'i') >= 2`.

Ejercicio 12 — Reino Unido y la tercera letra

Nivel: Avanzado · **Tablas:** `employees`, `departments`, `locations`, `countries`

Se pide. Todos los empleados que pertenezcan al Reino Unido (`United Kingdom`) y que tengan una «a» minúscula en la tercera posición del nombre.

Mostrar: nombre y apellido.

Idea. El país no está en `employees`: hay que viajar por `employees`, `departments`, `locations` y `countries`. La tercera letra se saca con `SUBSTR(texto, posición, largo)`.

Pasos.

1. `hr.employees` guarda el departamento del empleado (`department_id`).
2. `hr.departments` guarda la ubicación del departamento (`location_id`).
3. `hr.locations` guarda el país de la ubicación (`country_id`).
4. `hr.countries` traduce el código al nombre: `country_name = 'United Kingdom'`.
5. `SUBSTR(e.first_name, 3, 1) = 'a'` toma 1 carácter a partir de la posición 3 y lo compara con «a».

```
SELECT e.first_name,
       e.last_name
FROM hr.employees   e
JOIN hr.departments d ON d.department_id = e.department_id
JOIN hr.locations   l ON l.location_id   = d.location_id
JOIN hr.countries   c ON c.country_id    = l.country_id
WHERE c.country_name = 'United Kingdom'
  AND SUBSTR(e.first_name, 3, 1) = 'a';
```

Ojo. Con `JOIN` interno quedan fuera los empleados sin departamento: como no tienen ubicación, tampoco pertenecen a ningún país.

Variante. `e.first_name LIKE '__a%'` (dos guiones bajos, la «a» y cualquier cosa después) hace lo mismo que `SUBSTR`.